

Relatório de Decisões de Implementação

Sistema de Matchmaking para Jogos Online — C++

1 Estrutura de Dados Utilizada

O núcleo do sistema é a classe `Matchmaking`, que armazena os jogadores em um array estático de `Player` com capacidade máxima definida pela constante `MAX_PLAYERS = 100.000`:

```
Player players[MAX_PLAYERS];
int size;
```

O atributo `size` rastreia a quantidade atual de jogadores na fila. A inserção é sempre feita na posição `size` (final do array), e a remoção desloca os elementos subsequentes para preencher o espaço. Os algoritmos de ordenação `mergeSort` e `merge` foram implementados como funções livres que operam diretamente sobre o array via ponteiro, e declarados no cabeçalho da classe.

2 Algoritmos de Ordenação

2.1 Insertion Sort

O `sortByScoreInsertion` percorre o array da posição 1 até `size - 1`. Para cada elemento, desloca para a direita todos os jogadores anteriores com score estritamente maior, inserindo o elemento atual na posição correta. Por deslocar apenas elementos maiores, a ordem relativa entre elementos de mesmo score é preservada — tornando o algoritmo estável e garantindo o desempate por timestamp sem tratamento adicional.

2.2 Merge Sort

O `sortByScoreMerge` delega para a função recursiva `mergeSort(arr, left, right)`. O caso base ocorre quando `left ≥ right`. A cada chamada, o intervalo é dividido ao meio; após o retorno das duas metades ordenadas, a função `merge` as combina usando um array auxiliar de tamanho `right - left + 1`, intercalando os elementos em ordem crescente de score (com timestamp como desempate) e copiando o resultado de volta ao array original.

3 Uso da Ordenação na Formação de Grupos

O método `formGroup` assume que o array já está ordenado por score. Com essa garantia, qualquer janela de `groupSize` posições consecutivas tem seu menor score no primeiro elemento e seu maior score no último. A condição de validade do grupo reduz-se a:

```
waiting[i + groupSize - 1].getScore() - waiting[i].getScore() <= delta
```

O método itera `i` de 0 até `m - groupSize`, verificando cada janela consecutiva e retornando o primeiro grupo válido encontrado. Sem a ordenação prévia, seria necessário testar todas as combinações de `groupSize` jogadores, com custo combinatorial. Com o array ordenado, uma varredura linear é suficiente e correta.

4 Custo Computacional das Operações

Operação	Complexidade	Justificativa
<code>insert</code>	$O(1)$	Inserção na posição <code>size</code> ; apenas incrementa o contador.
<code>removePlayer</code>	$O(n)$	Busca linear pelo ID; desloca os elementos subsequentes uma posição à esquerda.
<code>sortByScoreInsertion</code>	$O(n^2)$	Dois laços aninhados; pior caso com array em ordem decrescente.
<code>sortByScoreMerge</code>	$O(n \log n)$	Divisão e conquista com profundidade $\log n$ e merge $O(n)$ por nível.
<code>formGroup</code>	$O(n)$	Varredura linear das janelas consecutivas, possível graças à ordenação prévia.
<code>getWaitingPlayers</code>	$O(n)$	Cópia linear do array interno para um novo array alocado dinamicamente.

Onde n é o número de jogadores atualmente na fila. Como `MAX_PLAYERS` é uma constante, os custos descrevem o comportamento em função do tamanho real da fila, não do array alocado.

5 Comparação Experimental: Insertion Sort vs. Merge Sort

5.1 Metodologia

Foram gerados arrays de jogadores com scores aleatórios para três tamanhos de entrada: 10.000, 50.000 e 100.000 jogadores. Os mesmos dados foram fornecidos a ambos os algoritmos em cada rodada, garantindo comparação justa. O tempo de execução foi medido com `std::chrono::high_resolution_clock`, considerando apenas a chamada ao método de ordenação.

5.2 Resultados

Tamanho (n)	Insertion Sort (ms)	Merge Sort (ms)
10.000	2060.62	46.907
50.000	52500.5	218.701
100.000	207772	469.208

5.3 Análise

Os resultados confirmam o comportamento esperado pela teoria. O insertion sort apresenta crescimento quadrático: ao dobrar o tamanho da entrada, o tempo tende a quadruplicar, tornando-se inviável para entradas grandes. O merge sort mantém crescimento consistente com $O(n \log n)$, sendo significativamente mais rápido já a partir de entradas moderadas. Para filas muito pequenas ou quase ordenadas, o insertion sort pode ser competitivo, pois seu custo se aproxima de $O(n)$ nesses casos. Para o contexto do sistema de matchmaking — com fila de até 100.000 jogadores — o merge sort é a escolha adequada.